

SQL Data Types

Complete Cheatsheet — MySQL / MariaDB

Sorted from Basics → Advanced → Professional use

BASICS — Start here

The types every beginner must know. Use these for 80% of real-world tables.

Integers

Type	Range / Size	Use when...	Example (SQL)
INT	-2.1B to 2.1B	Primary keys, general counts	<code>id INT NOT NULL AUTO_INCREMENT</code>
TINYINT	-128 to 127	Small flags, 0/1 booleans, age	<code>is_active TINYINT(1) DEFAULT 1</code>
BIGINT	-9.2×10^{18} to 9.2×10^{18}	Very large IDs, ms timestamps	<code>order_id BIGINT UNSIGNED</code>
BOOLEAN	Alias of TINYINT(1)	True/false fields	<code>is_verified BOOLEAN DEFAULT FALSE</code>

Strings

Type	Range / Size	Use when...	Example (SQL)
VARCHAR (n)	Up to 65 535 chars	Names, emails, short text	<code>email VARCHAR(255) NOT NULL</code>
CHAR (n)	Fixed n chars (max 255)	Fixed codes: country, hash prefix	<code>country_code CHAR(2)</code>
TEXT	~64 KB	Articles, comments, body text	<code>description TEXT</code>
ENUM (...)	Up to 65 535 members	Fixed allowed values	<code>status ENUM('pending', 'active', 'done')</code>

Dates & Numbers

Type	Range / Size	Use when...	Example (SQL)
DATE	1000-01-01 to 9999-12-31	Birthdays, deadlines (no time)	<code>dob DATE</code>
DATETIME	1000-01-01 to 9999-12-31	Full timestamps, not TZ-aware	<code>created_at DATETIME DEFAULT CURRENT_TIMESTAMP</code>
DECIMAL (p, s)	Exact, up to 65 digits	Prices, money — never use FLOAT	<code>price DECIMAL(10,2) NOT NULL</code>
FLOAT	~7 decimal digits	Approx. scores, ratios	<code>score FLOAT</code>

ADVANCED — Know these well

Needed for optimised, production-quality schemas.

More integers & fixed types

Type	Range / Size	Use when...	Example (SQL)
SMALLINT	-32 768 to 32 767	Port numbers, year values	<code>port SMALLINT UNSIGNED</code>
MEDIUMINT	-8.4M to 8.4M	Medium-range auto IDs, view counts	<code>views MEDIUMINT UNSIGNED DEFAULT 0</code>
DOUBLE	~15 decimal digits	GPS coords, scientific values	<code>latitude DOUBLE NOT NULL</code>
NUMERIC (p, s)	Alias for DECIMAL	Financial calculations	<code>tax NUMERIC(8,4)</code>

Time types

Type	Range / Size	Use when...	Example (SQL)
TIMESTAMP	1970 – 2038 (UTC)	Audit rows; auto-updates; TZ-aware	<code>updated_at TIMESTAMP ON UPDATE CURRENT_TIMESTAMP</code>
TIME	-838:59:59 to 838:59:59	Duration, schedule open/close times	<code>open_time TIME DEFAULT "09:00:00"</code>
YEAR	1901 to 2155	Academic year, car model year	<code>grad_year YEAR</code>

More string types

Type	Range / Size	Use when...	Example (SQL)
TINYTEXT	255 bytes	Very short notes	<code>tag TINYTEXT</code>
MEDIUMTEXT	~16 MB	Long articles, HTML pages	<code>content MEDIUMTEXT</code>
LONGTEXT	~4 GB	Huge logs, full-text dumps	<code>log LONGTEXT</code>
SET (...)	Up to 64 members	Multiple values from a fixed list	<code>tags SET("news", "tech", "sport")</code>

Bits & binary integers

Type	Range / Size	Use when...	Example (SQL)
BIT (n)	1–64 bits	Bitmask permissions, feature flags	<code>permissions BIT(8) DEFAULT b'00000000'</code>
INT UNSIGNED	0 to 4 294 967 295	Counts that never go negative	<code>download_count INT UNSIGNED DEFAULT 0</code>
BIGINT UNSIGNED	0 to 1.8×10^{19}	Very large positive-only IDs	<code>snowflake_id BIGINT UNSIGNED</code>

PROFESSIONAL — Power-user types

For specialised applications: GIS, binary storage, JSON APIs, and raw data.

▶ Binary / BLOB

Type	Range / Size	Use when...	Example (SQL)
BINARY (n)	Fixed n bytes (max 255)	Fixed hashes, raw UUIDs	<code>uuid BINARY(16)</code>
VARBINARY (n)	Up to 65 535 bytes	Variable encrypted tokens	<code>token VARBINARY(64)</code>
TINYBLOB	255 bytes	Tiny binary chunks	<code>icon TINYBLOB</code>
BLOB	~64 KB	Small images, thumbnails	<code>thumbnail BLOB</code>
MEDIUMBLOB	~16 MB	Audio clips, medium files	<code>audio MEDIUMBLOB</code>
LONGBLOB	~4 GB	Videos, full file backups	<code>video LONGBLOB</code>

▶ JSON (MySQL 5.7+)

Type	Range / Size	Use when...	Example (SQL)
JSON	Validated JSON doc	Flexible config, dynamic attributes, API payloads	<code>settings JSON</code>

▶ Spatial / GIS

Type	Range / Size	Use when...	Example (SQL)
GEOMETRY	Any spatial type	Generic spatial column	<code>shape GEOMETRY</code>
POINT	X,Y coordinate pair	GPS lat/lng, map pin	<code>coords POINT NOT NULL SRID 4326</code>
LINESTRING	Sequence of points	Roads, routes, paths	<code>route LINESTRING</code>
POLYGON	Closed shape	Delivery zones, map regions	<code>area POLYGON</code>
MULTIPOINT	Collection of points	Multi-location assets	<code>stops MULTIPOINT</code>
GEOMETRYCOLLECTION	Mixed spatial types	Complex GIS features	<code>features GEOMETRYCOLLECTION</code>

Quick Pick Guide

The right type for the most common real-world fields.

Storing...	Best type
Primary key (auto-increment)	<code>INT UNSIGNED NOT NULL AUTO_INCREMENT</code>
Money / price	<code>DECIMAL(10,2)</code> — never <code>FLOAT</code>
Email / username	<code>VARCHAR(255) NOT NULL</code>
Password hash (bcrypt)	<code>CHAR(60)</code> or <code>VARCHAR(255)</code>
True / false flag	<code>TINYINT(1) DEFAULT 0</code> or <code>BOOLEAN</code>

Storing...	Best type
Row creation timestamp	<code>TIMESTAMP DEFAULT CURRENT_TIMESTAMP</code>
Row update timestamp	<code>TIMESTAMP ON UPDATE CURRENT_TIMESTAMP</code>
Fixed status field	<code>ENUM('pending','active','done') DEFAULT 'pending'</code>
Long article / blog post	<code>TEXT</code> or <code>MEDIUMTEXT</code>
GPS coordinates	<code>POINT</code> or <code>DOUBLE (lat) + DOUBLE (lng)</code>
File stored in DB	<code>MEDIUMBLOB</code> or <code>LONGBLOB</code> (prefer file path in <code>VARCHAR</code>)
Flexible metadata / config	<code>JSON</code> (MySQL 5.7+)
UUID stored efficiently	<code>BINARY(16)</code> with <code>UUID_TO_BIN()</code> / <code>BIN_TO_UUID()</code>
Bitmask permissions	<code>BIT(8)</code> or <code>TINYINT UNSIGNED</code>

Key rules to remember

- Use `DECIMAL` for money — `FLOAT` and `DOUBLE` are approximate and will cause rounding errors.
- `TIMESTAMP` is timezone-aware and stored as UTC; `DATETIME` is stored as-is (no timezone).
- `TIMESTAMP` only goes up to 2038 (the Y2K38 problem) — use `DATETIME` for far-future dates.
- `VARCHAR` is almost always better than `TEXT` for indexed columns (`TEXT` cannot be fully indexed by default).
- Avoid storing files (images, videos) as BLOBs in production — store the file path in `VARCHAR` instead.
- `JSON` columns cannot be used as foreign keys or have default values other than `NULL`.
- Always use `UNSIGNED` for columns that should never be negative (counts, IDs, ages).